

Your Agent Escaped Without Escaping the Sandbox

Unit tests are dead.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–12 are the talk. Slides 13–21 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-00:50

Tell Tuesday as a scene. Pause before the last two sentences.

Here's a normal Tuesday. You ask your agent to lock the admin page. No login, no entry.

Say your agent is Opus 5.5. The best coding model there is. It's in a sandbox. No internet. The tests are locked.

A few minutes later, every test passes. And the admin page is wide open.

Unit tests are dead. Not because agents stopped writing them. Because agents write them.

2. All green. Door wide open.

00:50-01:35

Let the room look at the three results. Say the script line once, plainly.

We built this and recorded it. The tests say pass. The admin page lets anyone in. It should say no.

Quick note: the agent in our lab is a script. We played the attacker, so we can replay every step. We weren't asking whether a model would cheat. We were asking whether anyone would notice.

3. A test is a second opinion

01:35-02:45

Left side, then right side. Say the last line slowly.

Why did tests ever work? Two people. One wrote down what correct means. The other wrote the code. When they disagreed, the test failed. That's how you found bugs.

Now one agent writes both. The code and the test. And it wants one thing: green.

So a passing test is just the agent's own opinion. The test still runs. It's just not a second opinion anymore.

4. This isn't new

02:45-03:55

One breath per year. Nobody broke out of anything.

This isn't new.

2018: a bug-fixing program deleted the file with the right answers. Every test passed.

2024: Sakana's AI Scientist ran out of time. Instead of getting faster, it tried to give itself more time.

2025: on SWE-bench, agents, including a Claude model, dug through the project history and found the hidden answer.

Nobody broke out of anything. The answer was in reach. So they took it.

5. We built the setup that should stop this

03:55-04:45

Point to the agent's folder, then to BLOCKED.

So we built the setup that should stop this. Sandbox. No internet. The agent gets a folder to work in, with the code and the answer file. The checker is locked.

The agent tried to replace the checker. Blocked. Good. The checker ran. Fail. Correct. The code was broken.

6. Who did it?

04:45-06:00

One suspect at a time. Pause before the last line.

Then the tests passed. Same broken code. So who did it?

The code? Same file. Still lets everyone in.

The checker? Same file. The attempt to replace it was blocked.

The answer file? Changed. Turn them away became let them in. Every answer became let them in.

The agent didn't make the code pass the test. It made the test pass the code.

7. The flaw

06:00-07:10

Count the four lines. Stress the fourth.

Here's the flaw. The checker is locked. But it reads the answers from the agent's folder. The agent can change its folder. And writing tests is the agent's job. You can't take that away.

You can take away a key. You can take away a folder. You can't take away the pen. What you can change is whose copy counts.

And the change was right there in our logs. Nothing asked who made it.

8. The fix: separate the pens

07:10-08:20

Read the three rows top to bottom. Pause on REJECTED.

The fix: separate the pens. The agent writes the code. It can suggest tests. Something it can't touch decides which tests count.

We kept our own locked copy of the answers. Same broken code: rejected. Fixed code: accepted.

Will an AI reviewer fix this? I doubt it. The agent writes what the reviewer reads. We didn't test that. But it's the same problem.

9. Even our own test lied

08:20-09:30

Slow down for the first test. Then read the four rows in order.

Now the embarrassing part. Our release step lets one approved change out, once, to one place. We wrote a test to prove it blocks the wrong code.

It passed. But it tried the wrong code after the approval was already used up. Of course it was refused. The test proved nothing.

So we fixed the order. Wrong code first: refused. Wrong place: refused. Right code, right place: released. Again: refused.

A no only means something if the yes still works.

10. Every yes is a test

09:30-10:45

One breath per row: who says yes, our lab, the real case.

Tests aren't the only thing that says yes. Anything that trusts what the agent hands it is a test.

A service trusts a key. In our lab, a key opened another team's data. In 2025, a hacked build step called tj-actions printed secret keys into public logs.

The next job trusts a shared folder. Ours let one job change another's file.

An upload service trusts a request. Ours saved data into someone else's account. Last year, CamoLeak pulled private code out of GitHub's AI assistant through GitHub's own image links.

We recorded all four. And the fix for each, with the real job still working.

11. The benchmark builders just found out

10:45-11:30

Point at the two numbers. Read the quote exactly.

Last month, the people who build coding benchmarks found the same thing. Agents could reach the hidden tests. When they moved them out of reach, one model's score fell from 79 percent to 57.

Their fix: get the hidden tests out of the agent's folder. Separate the pens.

12. It escaped the test

11:30-12:15

Back to Tuesday. Say the title line, pause, then the question. Stop.

Back to Tuesday. The sandbox held. The checker held. Nothing broke out.

Your agent didn't escape the sandbox. It escaped the test.

So on Monday, pick one check your team trusts. Ask one question: could the agent have written what made it pass?

It's all on GitHub. Thank you.

Questions

13. What this experiment shows

Questions

Use only in questions.

We tested whether the setup allows these moves, and whether the fix stops them. We did not measure how often a model would try, and we did not run Opus 5.5; it's the example in the story. The five cases on the right are the whole test. Passing them proves those five cases and nothing more.

14. The key reached another team's data

Questions

Use only in questions.

A key limited to the task fixed it, and the task could still read its own input. A shorter expiry alone doesn't help: a broad key can reach everything it covers until it expires. The service has to check what the key is for.

15. The shared file changed on the host

Questions

Use only in questions.

The change stayed on the host, waiting for the next job. We didn't run that next job, so that part is what would happen, not something we recorded. With the folder read only, the same write was refused, and the agent could still write its own output.

16. The upload landed in another account

Questions

Use only in questions.

Reaching a service doesn't mean you may do anything there. The service really saved the data, so this is delivery, not just an attempt. In the fix, a sender outside the agent sends one fixed report to the right account, refuses anything extra, and the agent can't reach the service directly.

17. A bug in our old release step

Questions

Use only in questions.

Our old release step trusted an old fingerprint after the code changed. Triggering it needed direct access to our own controller, and we did not show the agent could get that. The new release step computes the fingerprint from the exact code it releases.

18. What we recorded

Questions

Use only in questions.

These count checks, not bugs found. The 49 are regression tests for our own code, not sandbox tests. The old bug is one reproduction of a mistake in our earlier release step.

19. Run it yourself

Questions

Use only in questions.

You need a throwaway Linux machine with bubblewrap, Python 3.10 or later, and make. On any computer, make test and make factory run the local checks, without a sandbox. GitHub Actions builds the PDFs on every change. No cloud account needed.

20. What the approval covers

Questions

Use only in questions.

The controller checks the results, then signs its decision together with the code, the answers and the destination, using a key only it holds. The approval names the task and the run, expires, and works once. The raw results themselves are not signed.

21. Sources

Questions

Use only in questions.

These are the sources for every real-world case in the talk. Each one is someone else's report. We don't claim their numbers as ours.